JavaScript Variable Declarations: var, let, and const

Introduction

JavaScript provides three keywords for declaring variables: `var`, `let`, and `const`. Understanding their differences is critical for writing predictable, maintainable code. Each keyword exhibits distinct behavior regarding hoisting, scope, redeclaration, and reassignment.

Hoisting Behavior

Hoisting is JavaScript's mechanism of moving variable and function declarations to the top of their scope during the compilation phase, before code execution begins. This behavior differs significantly between `var`, `let`, and `const`.

Understanding var Hoisting

The term `var` is derived from "variant," not "variable," reflecting its flexible and often unpredictable nature. When you declare a variable with `var`, the declaration is hoisted to the top of its scope, but the initialization remains at the original location.

```
console.log(a);  // What will this output?
var a = 45;
console.log(a);  // What about this?
```

Output:

`undefined`
`45`

The first `console.log(a)` outputs `undefined` rather than throwing an error because the declaration `var a` is hoisted to the top of the scope. However, the assignment `a = 45` remains at line 2. JavaScript effectively interprets the code as:

Powered by SPRK Technologies

```
var a;           // Declaration hoisted to top
console.log(a);  // undefined (declared but not initialized)
a = 45;          // Assignment happens here
console.log(a);  // 45
```

Temporal Dead Zone (TDZ)

Variables declared with `let` and `const` are also hoisted, but they remain in a "Temporal Dead Zone" from the start of the block until the declaration is encountered. Accessing them during this period results in a ReferenceError.

```
console.log(b);  // Attempt to access before declaration
let b = 35;
```

Error:

`ReferenceError: Cannot access 'b' before initialization`

```
console.log(c);  // Attempt to access before declaration
const c = 85;
```

Error:

`ReferenceError: Cannot access 'c' before initialization`

The Temporal Dead Zone exists from the beginning of the block until the line where the variable is declared and initialized. During this period, the variable exists in the scope but cannot be accessed.

Note:

The TDZ is a design choice that prevents the use of variables before they are properly initialized, reducing potential bugs and making code behavior more predictable.

Reassignment and Mutability

Understanding const Immutability

The `const` keyword creates a constant reference to a value. Once initialized, the binding cannot be reassigned to a different value.

```
const c = 85;
c = 100;  // Attempt to reassign
```

Error:

`TypeError: Assignment to constant variable.`

While the reference is immutable, it's important to understand that `const` does not make objects or arrays immutable—only the binding itself:

```
const arr = [1, 2, 3];
arr.push(4);        // Allowed - modifying contents
console.log(arr);

arr = [5, 6, 7];    // Not allowed - reassigning reference
```

Output:

`[1, 2, 3, 4]`

Error:

`TypeError: Assignment to constant variable.`

Powered by SPRK Technologies

Reassignment Capabilities

Keyword	Reassignment	Example
`var`	Allowed	`var x = 1; x = 2;`
`let`	Allowed	`let y = 1; y = 2;`
`const`	Not Allowed	`const z = 1; z = 2; // Error`

Redeclaration Rules

Redeclaration refers to declaring a variable with the same name multiple times within the same scope. The behavior varies significantly across the three keywords.

var Allows Redeclaration

```
var a = 45;
console.log(a);

var a = "Hello";  // Redeclaration allowed
console.log(a);
```

Output:

```
45
Hello
```

The ability to redeclare variables with `var` can lead to accidental overwrites, especially in large codebases where variable names might collide.

Powered by SPRK Technologies

let and const Prevent Redeclaration

```
let b = 35;
let b = "Hi";  // Attempt to redeclare
```

Error:

`SyntaxError: Identifier 'b' has already been declared`

```
const c = 85;
const c = "Hello again";  // Attempt to redeclare
```

Error:

`SyntaxError: Identifier 'c' has already been declared`

Note:

The prevention of redeclaration with `let` and `const` is a safety feature that catches potential naming conflicts at parse time, before the code executes.

Scope Differences

Scope determines the visibility and lifetime of variables. JavaScript has function scope, block scope, and global scope. The choice of declaration keyword directly impacts how variables are scoped.

Block Scope vs Function Scope

Block scope is defined by curly braces `{}`, such as those used in if statements, loops, and standalone blocks. Function scope is limited to the function in which a variable is declared.

Powered by SPRK Technologies

```
if (true) {
    var test = 10;       // Function scoped
    let test2 = 20;      // Block scoped
    const test3 = 30;    // Block scoped
}

console.log(test);   // Accessible
console.log(test2);  // Not accessible
console.log(test3);  // Not accessible
```

Output:

`10`

Error:

`ReferenceError: test2 is not defined`

Error:

`ReferenceError: test3 is not defined`

Why var is Function Scoped

Variables declared with `var` are scoped to the nearest function, or to the global scope if declared outside any function. Block-level constructs like if statements and loops do not create new scopes for `var`.

```
function example() {
    if (true) {
        var functionScoped = "I'm accessible throughout the function";
    }
    console.log(functionScoped);  // Works fine
}

example();
```


Powered by SPRK Technologies

Output:

```
I'm accessible throughout the function
```

Block Scope with let and const

Variables declared with `let` and `const` are block-scoped, meaning they only exist within the block in which they are declared.

```
if (true) {
    let test2 = 20;
    const test3 = 30;
    console.log(test2);  // Accessible within block
    console.log(test3);  // Accessible within block
}

// Outside the block - these are different variables
let test2 = 100;
console.log(test2);
```

Output:

```
20
30
100
```

Notice that declaring `let test2 = 100` outside the block is valid because it's in a different scope. The inner `test2` and outer `test2` are completely separate variables.

Practical Scoping Example

```
for (var i = 0; i < 3; i++) {
    setTimeout(() => console.log("var loop:", i), 100);
}
```


Powered by SPRK Technologies

```
for (let j = 0; j < 3; j++) {
    setTimeout(() => console.log("let loop:", j), 100);
}
```

Output:

```
var loop: 3
var loop: 3
var loop: 3
let loop: 0
let loop: 1
let loop: 2
```

The `var` version outputs 3 three times because `i` is function-scoped and shared across all iterations. By the time the setTimeout callbacks execute, the loop has completed and `i` equals 3.

The `let` version outputs 0, 1, 2 as expected because `j` is block-scoped. Each iteration creates a new binding of `j`, capturing the current value.

Comparison Table

Feature	`var`	`let`	`const`
Scope	Function	Block	Block
Hoisting	Yes (initialized as undefined)	Yes (TDZ until declaration)	Yes (TDZ until declaration)
Redeclaration	Allowed	Not Allowed	Not Allowed
Reassignment	Allowed	Allowed	Not Allowed
Initialization Required	No	No	Yes
Global Object Property	Yes (if declared globally)	No	No

Powered by SPRK Technologies

Best Practices

Modern JavaScript Recommendations

> **Warning:**
>
> Avoid using `var` in modern JavaScript development. The unpredictable behavior of function scoping and hoisting can lead to subtle bugs that are difficult to diagnose.

Preferred approach: Use `const` by default. This creates immutable bindings and signals intent that the variable should not be reassigned.

When to use let: Use `let` only when you need to reassign the variable, such as in loops or algorithms that require mutation.

```
// Good: Use const for values that won't change
const MAX_USERS = 100;
const API_URL = "https://api.example.com";

// Good: Use let for values that will change
let counter = 0;
for (let i = 0; i < 10; i++) {
    counter += i;
}

// Avoid: Don't use var
var oldStyle = "Don't do this";
```

Declaration Without Initialization

```
let uninitializedLet;
console.log(uninitializedLet);

const uninitializedConst;  // This will cause an error
```

> **Output:**

Powered by SPRK Technologies

```
undefined
```

Error:

```
SyntaxError: Missing initializer in const declaration
```

Variables declared with `let` can be declared without initialization and will default to `undefined`. However, `const` requires initialization at the point of declaration.

Global Object Pollution

When declared in the global scope, `var` creates properties on the global object (window in browsers), while `let` and `const` do not.

```
var globalVar = "I'm on window";
let globalLet = "I'm not on window";
const globalConst = "I'm not on window either";

console.log(window.globalVar);    // In browser
console.log(window.globalLet);    // In browser
console.log(window.globalConst);  // In browser
```

Output:

```
I'm on window
undefined
undefined
```

This behavior makes `var` particularly problematic in global scope, as it can unintentionally overwrite properties on the global object or create naming conflicts with built-in browser APIs.

Powered by SPRK Technologies

Summary

The evolution from `var` to `let` and `const` represents a significant improvement in JavaScript's variable declaration system. Modern JavaScript development prioritizes predictability, block scoping, and immutability by default.

Key Takeaways:

`var` is function-scoped, hoisted with undefined initialization, allows redeclaration, and creates global object properties. These characteristics make it unsuitable for modern development.

`let` is block-scoped, subject to the Temporal Dead Zone, prevents redeclaration, allows reassignment, and does not pollute the global object. Use it for variables that need to change.

`const` shares all characteristics with `let` but additionally prevents reassignment and requires initialization. Use it by default for all declarations unless reassignment is necessary.

> **Note:**
>
> The principle of "const by default, let when necessary, never var" leads to code that is easier to reason about, less prone to bugs, and more maintainable over time.